

AI-Driven Automated Answer Sheet Evaluation System

Complete Project Workflow

BMS Institute of Technology & Management
Department of Computer Science and Engineering
Major Project Phase 2 — 2025–26

P Charan Chandra | R T Kesav Reddy | Sai Charan M M | Nunna Uma Shankar
Under the guidance of Dr. Gireesh Babu C N

This document describes the complete end-to-end workflow of the AI-Driven Automated Answer Sheet Evaluation System — from teacher login through AI pipeline processing to final student result. Every layer of the pipeline is explained with the exact tools, techniques, and datasets used.

1. Who Uses the System

Teacher / Examiner	Uploads model key answers, uploads student answer sheets one by one, reviews flagged evaluations, overrides scores if needed, approves results for student release.
AI Pipeline (Automated)	Runs automatically after each upload. Preprocesses the image, detects structural annotations, extracts text, evaluates meaning, computes partial credit, generates feedback, and flags low-confidence results.
Student	Logs into the student portal after the teacher approves results. Views their score, per-question concept coverage, and three-section feedback.

2. Phase 1 — Teacher Setup

Teacher sets up the exam before any evaluation begins

1	Register and login Teacher creates an account with full name, email, department, username, and password. Spring Security stores a BCrypt-hashed password (12 rounds) in MySQL. On login, Spring Security verifies credentials and creates a secure HTTP session. [Spring Security 6] [BCrypt 12 rounds] [MySQL users table] [HTTP Session]
---	---

2

Create an exam and add questions

Teacher creates an exam (e.g. "DBMS Unit 3"). For each question, the teacher enters: question text, maximum marks, and the model key answer. The model answer can be typed directly or uploaded as a PDF/image. This model answer is saved persistently and reused for every student sheet of that question until the teacher explicitly changes it.

[Exam entity] [Question entity] [Model answer stored as text] [Persistent until changed]

Key design point — model answer is set ONCE

The teacher uploads the model key answer once per question. It remains active for ALL student sheets of that question. The teacher only changes it if the answer needs updating. This means the LLM concept extraction (which reads the model answer) also runs once per question and its result is cached — saving significant computation time.

3

Add students

Teacher adds students manually or imports via CSV file (USN, name, email). Students are linked to the exam. Each student gets a portal account to view their results after the teacher approves them.

[Student entity] [CSV bulk import (Apache Commons CSV)] [Linked to exam]

3. Phase 2 — Upload and AI Pipeline

Teacher uploads one student sheet → 8-layer AI pipeline runs automatically

Step 1 — Teacher uploads a scanned answer sheet

Teacher selects a student from the dropdown and uploads their scanned answer sheet (JPEG, PNG, or PDF). Spring Boot saves the file to disk with a UUID filename to prevent path traversal attacks. The system creates an AnswerSheet record in MySQL and triggers the AI pipeline as a background task using @Async — so the upload response is instant and the teacher sees a live progress bar while evaluation runs.

Tech: Spring MultipartFile UUID filename AnswerSheet entity @Async background task Spring SseEmitter (live progress)

The 8-Layer AI Pipeline (Python Service on port 8001)

Spring Boot calls the Python FastAPI AI service via WebClient/RestTemplate. The service processes the image through 8 sequential layers:

Layer 1 — Image Preprocessing

The scanned image is cleaned so OCR can work accurately. Four operations run in sequence:

Greyscale + Gaussian blur Remove colour channels. Blur smooths ink irregularities and reduces high-frequency noise.	Adaptive thresholding Convert to black and white. Local window method handles pages with uneven lighting — no single global threshold needed.
Deskewing via Hough transform Detect dominant text line angle. Apply compensating rotation. Even 2-degree skew drops Tesseract accuracy by 15%.	CLAHE contrast normalisation Contrast Limited Adaptive Histogram Equalisation boosts local contrast so faint ink becomes clearly readable.

Tech: OpenCV (cv2) NumPy GaussianBlur adaptiveThreshold HoughLinesP CLAHE

Layer 2 — Document Segmentation

The cleaned page is split into individual answer blocks — one per question. Horizontal projection profiles count foreground pixels per row. Valleys in this profile (rows with very few ink pixels) are whitespace gaps between answers. `scipy.signal.find_peaks` locates these valleys. Each block is tagged with a Question ID from the exam template stored in MySQL.

Why segmentation is critical

Without segmentation, the LLM would grade all questions together against one rubric. Segmentation ensures Q1 answer is always graded against Q1 model answer and Q2 against Q2 — no cross-contamination.

Tech: NumPy projection profiles cv2.findContours `scipy.signal.find_peaks` Question ID tagging

Layer 3 — Structural Element Detection (Core Novelty)

This is what makes this project unique. Every other automated grading system treats a handwritten answer as a flat block of text. This layer reads the page the way a human examiner would — understanding visual annotations that convey student intent.

Strike-throughs → DELETION Hough line transform detects near-horizontal lines (angle < 15°) overlapping text regions. Text beneath a confirmed strike-through is tagged DELETED and excluded from OCR output. The student's corrected version above or beside the deletion is used instead.	Underlines → EMPHASIS Sobel horizontal edge detection finds strong horizontal edges immediately below text baselines. Underlined words are tagged as IMPORTANT and given higher weight in SBERT concept matching — the student signalled these words matter.
Arrows → INSERTION POINTER Convexity defect analysis finds arrow shapes in contours. Arrow tip coordinates point to the insertion position in the main text. Margin text connected by the arrow is integrated at that exact position during reconstruction — not discarded.	Output: annotation list Each detection returns: type (DELETION / EMPHASIS / INSERTION), bounding box, confidence score, and linked text region ID. This structured list feeds directly into Layer 5 reconstruction.

Tech: cv2.HoughLinesP (strike-through) cv2.Sobel (underline) cv2.convexityDefects (arrow) Text region masking Confidence thresholding

Layer 4 — Adaptive OCR

Each answer block (with DELETED regions already excluded) goes through a quality assessment. Three metrics are computed: stroke density (proportion of dark pixels), Laplacian variance (sharpness of edges), and pixel variance (consistency of ink weight). These combine into a quality score from 0 to 1.

Quality >= 0.60 → Tesseract direct PSM 6 (uniform text block mode). Fast and accurate for clean handwriting. Returns text plus per-character confidence scores that feed the confidence scoring in Layer 8.	Quality < 0.60 → Enhanced path Morphological operations (dilation kernel 2x2, morphological closing) reconnect broken strokes. CLAHE re-applied. Tesseract PSM 11 (sparse text mode) runs on enhanced image. If confidence still below 40%, block is flagged for human review.
--	---

Tech: Tesseract 5.x pytesseract cv2.Laplacian cv2.morphologyEx PSM 6 / PSM 11

Layer 5 — Text Reconstruction

All extracted text fragments are assembled into a clean, logical answer string. Three operations execute in strict order:

Step 1	Remove deletions	Any fragment whose bounding box overlaps a DELETION annotation is removed from the working set.
Step 2	Integrate margin text	Arrow-linked margin fragments are inserted into main text at the position indicated by the arrow tip, found using scipy KD-tree nearest neighbour search.
Step 3	Sort reading order	Remaining fragments sorted by vertical position then horizontal position (top to bottom, left to right). Output: one clean answer string per question.

Tech: scipy.spatial.KDTree Spatial IoU overlap Position-based sorting

Layer 6 — LLM Concept Extraction and Coverage Check

This is where meaning is understood. Llama 3.1 8B runs fully offline via Ollama (no internet, no API key, no cost per call). Two distinct steps happen:

Step A — Extract concepts from model answer (runs once per question, result cached) LLM reads the teacher's model key answer and extracts 5-8 key concepts with importance weights that sum to 1.0. Example output: "1NF requires atomic values (0.30), 2NF requires full functional dependency (0.25), 3NF no transitive dependency (0.25), BCNF every determinant is a candidate key (0.20)". Temperature = 0.1 for deterministic output. JSON mode enforced.	Step B — Check coverage in student answer (runs per student, triple-pass) LLM reads the student's reconstructed answer plus the concept list. For each concept it returns: covered / partial / missing, with a one-sentence evidence quote. Runs 3 times independently with fresh context each time. Majority vote per concept gives a stable, consistent coverage result — reducing single-pass variance significantly.
---	--

The LLM does NOT assign marks — it only reads meaning

The LLM's job is purely semantic judgment: "did the student communicate this concept?" It does not decide scores. Marks are computed mathematically by the SBERT layer from the coverage results. This separation makes the system explainable and auditable.

Tech: Llama 3.1 8B via Ollama Temperature 0.1 JSON output mode Triple-pass majority vote ThreadPoolExecutor (parallel calls)

Layer 7 — SBERT Partial Credit Scoring

Sentence-BERT (all-MiniLM-L6-v2, 22M parameters, 384-dimensional embeddings) verifies the LLM's coverage judgments using cosine similarity. This prevents LLM hallucination — if the LLM claims a concept was "covered" but cosine similarity is below 0.45, the status is automatically downgraded to "partial". Marks are then computed:

Covered	SBERT similarity >= 0.72 AND LLM says covered	Full weight x max_marks for this concept
Partial	Similarity 0.50-0.72 OR LLM says partial OR SBERT corrects overclaim	0.5 x weight x max_marks for this concept
Missing	Similarity < 0.50 AND LLM says missing	0 marks for this concept
Underlined words	Words detected as underlined in Layer 3	+10% weight boost in similarity computation

Tech: sentence-transformers all-MiniLM-L6-v2 numpy cosine similarity Weighted partial credit formula

Layer 8 — Feedback Generation and Confidence Scoring

Two things happen in parallel at the final layer:

Feedback generation (Llama 3.1 8B, Temperature 0.4) Three-section student feedback is generated from the coverage results. Section 1 — What was good: specifically names which concepts the student covered correctly. Section 2 — What was missing: explains each missing or partial concept clearly — teaches the concept rather than just saying "wrong". Section 3 — How to improve: one specific, actionable study suggestion.	Confidence score computation Composite confidence = 0.35 x OCR quality + 0.40 x LLM triple-pass agreement + 0.25 x SBERT verification rate. Score >= 0.80: green (published automatically). Score 0.65-0.80: orange (published with note). Score < 0.65: red (flagged on teacher dashboard for manual review before publishing).
--	---

Tech: Llama 3.1 8B Temperature 0.4 Weighted composite confidence MySQL evaluation table Review flag

4. Phase 3 — Result Sheet

Teacher reviews → approves → student sees result

What the result sheet shows for every evaluated answer sheet:

*	Score Final score per question (e.g. 12 / 15 marks), computed from weighted partial credit scores of all concepts
*	Concept bars Each concept shown as covered / partial / missing with a percentage bar and SBERT similarity value
*	Annotation summary N strike-throughs detected and ignored, N margin insertions integrated, N underlined keywords given weight boost

- * **Confidence**
Green / Orange / Red confidence indicator with OCR, LLM agreement, and SBERT clarity subscores shown separately
- * **Feedback**
Three-section feedback: what was good, what was missing (with concept explanation), how to improve
- * **Override**
Teacher can change score, add personal comment, and approve — all before student can see the result

After teacher approves

Student logs into the student portal and sees their complete result — score, per-question breakdown, concept coverage bars, and all three sections of feedback. The result is locked after approval. Re-evaluation requires the teacher to explicitly trigger a new pipeline run.

5. Datasets Used in the Project

IAM Handwriting Database

Use: CV pipeline calibration | Size: 1,539 pages from 657 writers

Calibrate Tesseract PSM modes and test preprocessing pipeline on diverse handwriting styles.

Download: kaggle.com/datasets/naderabdelghany/iam-handwritten-forms-dataset

CVL Handwriting Dataset (TU Graz)

Use: Quality scorer calibration | Size: 1,604 pages from 310 writers

Calibrate Laplacian variance and stroke density thresholds for the adaptive OCR quality scorer.

Download: cvl.tuwien.ac.at

FUNSD Dataset

Use: Segmentation testing | Size: 199 annotated scanned forms

Test projection profile segmentation on complex noisy document layouts similar to exam sheets.

Download: guillaumejaume.github.io/FUNSD

SROIE Dataset (ICDAR 2019)

Use: Structural detection testing | Size: 1,000 scanned receipts

Test Hough line strike-through detection using real horizontal line features in receipt documents.

Download: kaggle.com/datasets/urbikn/sroie-datasetv2

STS Benchmark (STSb)

Use: SBERT threshold validation | Size: 8,628 sentence pairs with human scores

Validate cosine similarity thresholds (HIGH=0.72, LOW=0.50) against human similarity judgments.

Download: huggingface.co/datasets/sentence-transformers/stsb

ASAP-SAS Dataset (Kaggle)

Use: LLM evaluation benchmarking | Size: 17,043 short student answers

Short exam-style answers with rubrics and human scores. Used to compute Pearson r between system and human grader scores.

Download: kaggle.com/c/asap-sas/data

Own manually graded sheets

Use: Final validation (ground truth) | Size: 20-50 real answer sheets

Teacher grades sheets manually. System output compared against these to compute final Pearson correlation coefficient.

Download: Created during Days 35-38 testing

6. AI Models Used

Llama 3.1 8B (Meta)

Used for: Concept extraction + Coverage check + Feedback generation

Instruction-tuned, 8 billion parameters, trained on 15 trillion tokens. Strong JSON output mode. Runs fully offline on 16GB RAM via Ollama. Temperature 0.1 for grading, 0.4 for feedback. No API key. No internet. No cost per call.

Install: `ollama pull llama3.1:8b` (downloads ~4.7GB)

all-MiniLM-L6-v2 (sentence-transformers)

Used for: Concept embedding + Cosine similarity + Partial credit verification

22 million parameters, 384-dimensional embeddings, trained on 1 billion sentence pairs. Produces 14,000+ embeddings per second on CPU. Downloaded automatically via pip install sentence-transformers.

Install: `pip install sentence-transformers` (auto-downloads ~90MB)

Tesseract 5.x OCR

Used for: Text extraction from clean handwriting blocks

Open-source OCR engine maintained by Google. LSTM-based recognition engine in version 5. Pre-trained on IAM handwriting dataset and synthetic data. Used with PSM 6 for clean blocks and PSM 11 for degraded blocks after morphological enhancement.

Install: `sudo apt install tesseract-ocr pip install pytesseract`

7. Complete Workflow — One Sentence

The workflow model is designed to process student sheets. It opens a clean and segment the page for structural detection (headers, sections, paragraphs, margin, arrow, text, and flags) and then extracts text. The script extracts text from LLM reads model answer to extract key concepts, checks their readability and checks which concepts are covered (multiple pass matching) using all-MiniLM-L6-v2 sentence embeddings via cosine similarity and computes weighted partial credit using Llama-generated feedback. Feedback confidence score decides whether teacher review is needed. Teacher approves or student sees their score, concept breakdown, and feedback.

